



VNRVJiet

VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF
ENGINEERING & TECHNOLOGY

AUTONOMOUS INSTITUTION



AI-WEEK

Vibe Coding Hackathon

Semantic Entropy File System

Team Details :

Team Name: **Tech Titans**

Team Member Name and Phone Number:

R.Anjana (CSE-Data Science)	-	9346291002
M.Soni (CSE)	-	9885590390
Hema.NS (CSE)	-	6305028626
N.Swathi (CSE)	-	7730821531

Problem Statement :

Traditional file management relies on **rigid, manual hierarchies** of folders and sub-folders that quickly become **disorganised and inefficient**. The **Semantic Entropy File System (SEFS)** challenge tasks you with developing a **self-organising file manager** that replaces **static directories** with a **dynamic semantic layer** integrated directly into the **operating system**. Instead of requiring users to **categorise files manually**, the system must **autonomously analyse the content** of documents and project them into a **live file structure** (for example: **Graph, FlowChart, Spatial Map**, etc.). In this environment, files with **high conceptual similarity** automatically converge into **meaningful folders**, while **unrelated documents** remain separated.

All **semantic processing** must operate on files located within a **single monitored root folder**. Within this **root directory**, the system must **dynamically organise files** into multiple **semantic folders** based on their **content**.

Each folder should contain only the files that **conceptually belong together**. Unlike **conventional file managers**, this **semantic structure** must function at the **OS level** rather than as a **standalone visualisation tool**. The system should **monitor file operations** such as **creation, modification, renaming, and deletion**, and continuously **synchronise semantic organisation** with the **operating system's file view**. When **semantic relationships change**, the **OS-level folder structure** must update accordingly.

The core difficulty lies in maintaining a “**living**” **semantic layout**: as **new data is introduced** or **existing content changes**, the system must **re-calculate file positioning**, update the **OS-integrated folder structure**, and apply **live changes in real time** without breaking **file integrity** or disrupting **normal system operations**.

Proposed Solution - Semantic Entropy File System

Objective

To develop a **self-organising OS-level file management system** that dynamically groups files based on semantic meaning instead of manual folder hierarchies.

What it Solves

Traditional file systems organize data using fixed folders and paths. Users must remember where files are and how they're categorized. A semantic file system organizes and retrieves files based on *content meaning and context*, not just location.

How the Solution Works

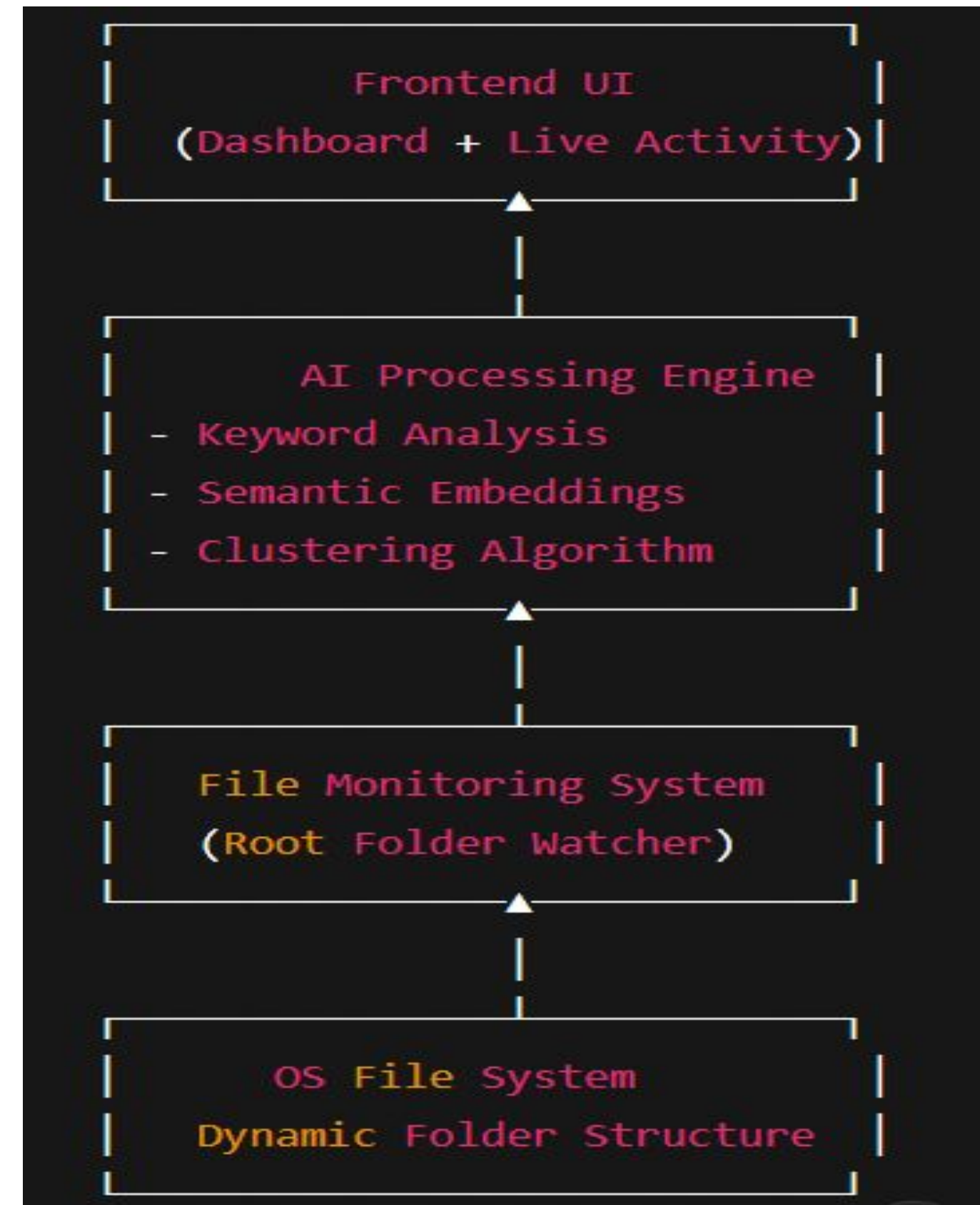
1. **Monitor a single root directory** continuously.
2. When a file is:
 - Created

- Modified
- Renamed
- Deleted

→ System automatically **detects the change**.

2. **Extract text from PDF/Text files.**
3. **Generate semantic embeddings (AI vector representation).**
4. **Calculate similarity between files.**
5. **Automatically:**
 - Create semantic folders
 - Move files into appropriate folders
 - Recalculate when content changes
6. **Reflect updates directly in OS folder structure.**
7. **Update live 2D visualization interface.**

System Architecture :



- **Login / Sign Up**
 - Users can **sign up** with email and password or **login** using Google.
 - Ensures a personalized workspace.
- **Main Page / Dashboard**
 - After login, the user is taken to the **main page**.
 - The main page shows **all uploaded files** and **existing semantic groups** (clusters/folders).
- **File Upload**
 - Users can **upload files** (PDFs or text files).
 - The system automatically **analyses the content**.
- **Automatic Grouping / Clustering**
 - Files are **automatically grouped into semantic clusters** based on content similarity.
 - Each cluster represents a **folder or category** with files of similar meaning.

- **File Operations**

- Users can perform standard operations:

- **Read / Open files**
- **Edit files**
- **Rename files**
- **Delete files**

- Any operation is **synchronized with the semantic system**, updating clusters in real time.

- **Interactive Visual Interface**

- Files are displayed as **nodes in a 2D interactive graph**.

- Users can:

- **Hover** over a file to see metadata (name, date, type, summary).
- **Click** to open files.
- See files **grouped visually by semantic similarity**.

- Layout **updates automatically** when new files are added or existing files are modified.

- **OS-Level Synchronisation**

- Semantic folders exist **physically in the root directory**.

- Any **manual change in the OS** (move, rename, delete) triggers **re-analysis** and updates clusters automatically.

Tech Stack :



Frontend (User Interface)

These files are part of the **React UI** that users interact with:

- **App.tsx** – The main React component that loads the whole frontend and ties all views together.
- **index.tsx** – Entry point that renders the React app into the web page.
- **index.html** – Base HTML page that hosts the React app.
- **Auth.tsx** – UI for user **authentication** (login/signup).
- **FileDetailModal.tsx** – Popup/modal that shows details about a selected file.
- **FileNodeGraph.tsx** – Visual graph showing relationships between files (nodes and links).
- **FolderView.tsx** – Component that shows a **folder's content view** (files and subfolders).
- **OSMonitor.tsx** – Shows some **system/OS status** (CPU/RAM, etc.) within the UI.



Services / Logic

Things that handle data or backend interaction:

- **geminiService.ts** – Manages communication with the **Gemini API** or backend logic (AI model calls, semantic processing).

Thank You